



COLLEGE CODE : 9505

COLLEGE NAME : Dr. Sivanthi Aditanar College of Engineering

DEPARTMENT : Information Technology

STUDENT NM-ID : 0C4C4D8D317DF6A8330468F3CB5D95C8

ROLL NO : 11234055

DATE : 06/10/2025

Completed the project named as

Phase 5. Project Demonstration & Documention

NAME : To-Do List Application

SUBMITTED BY,

NAME : A.Uchini Makali

MOBILE NO : 8525047373

1. Final Demo Walkthrough

Your project is a **To-Do List App** with the following features:

- Add, edit, and delete tasks.
- Set task **priority** (Low, Medium, High).
- Set **due dates** with overdue task highlighting.
- Mark tasks as **completed** (strikethrough effect).
- **Search filter** for quick task lookup.
- **Dark/Light** mode toggle.
- **Local Storage** support (tasks persist after reload).

2. Project Report

1. Project Title

To-Do List Application

2. Abstract

The To-Do List Application is a web-based productivity tool that helps users efficiently manage their daily tasks. The system allows users to add, edit, delete, and search tasks with customizable priority levels and due dates. It includes a dark/light mode toggle, ensuring a pleasant user experience across devices. The application stores data locally using the browser's Local Storage, making it lightweight and accessible without any backend server.

3. Objectives

- To develop a simple, interactive, and user-friendly To-Do List web application.
- To provide efficient task management with priority and deadline tracking.
- To enhance productivity through easy task visualization and control.
- To improve user experience with features like dark mode and search filtering.
- To ensure persistence using LocalStorage for saving tasks locally.

4. Technologies Used

Category	Tools / Technologies
Frontend	Bootstrap 5
UI Framework	Bootstrap 5
Storage	Browser LocalStorage
Deployment	Netlify / Vercel
Version Control	Git & GitHub

5. Features

- ✓ Add, delete, and edit tasks
- ✓ Assign task priority (Low, Medium, High)
- ✓ Add due date for each task

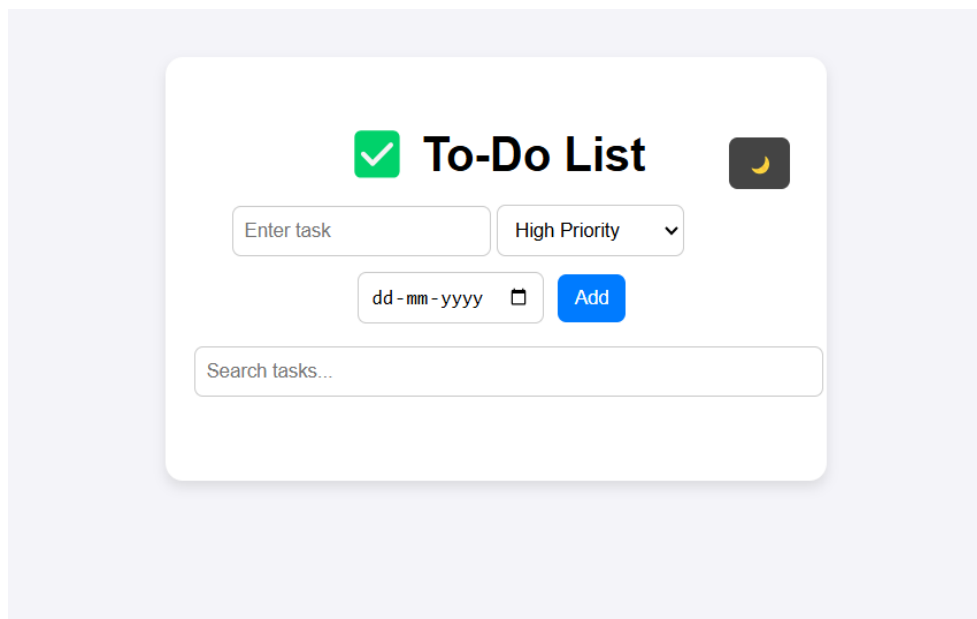
- ✓ Search bar for quick task lookup
- ✓ Dark / Light mode toggle
- ✓ Responsive UI with Bootstrap
- ✓ Task persistence using LocalStorage

6. Enhancements (Phase 4 Additions)

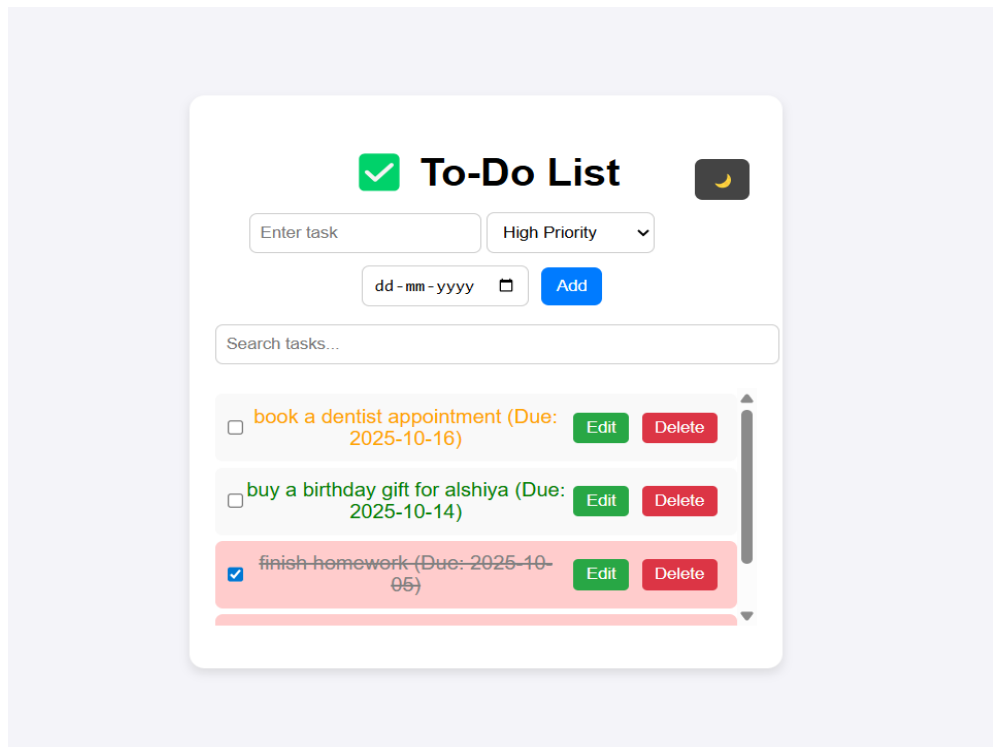
- Added **Dark/Light Mode Toggle**.
- Implemented **Search Filter** for easy navigation.
- Improved **UI/UX Design** using Bootstrap 5.
- Enhanced **Performance & Responsiveness** for all screen sizes.
- Added **Priority Dropdown** and **Due Date Calendar Input**.

Screenshots

1. Task Entry Form

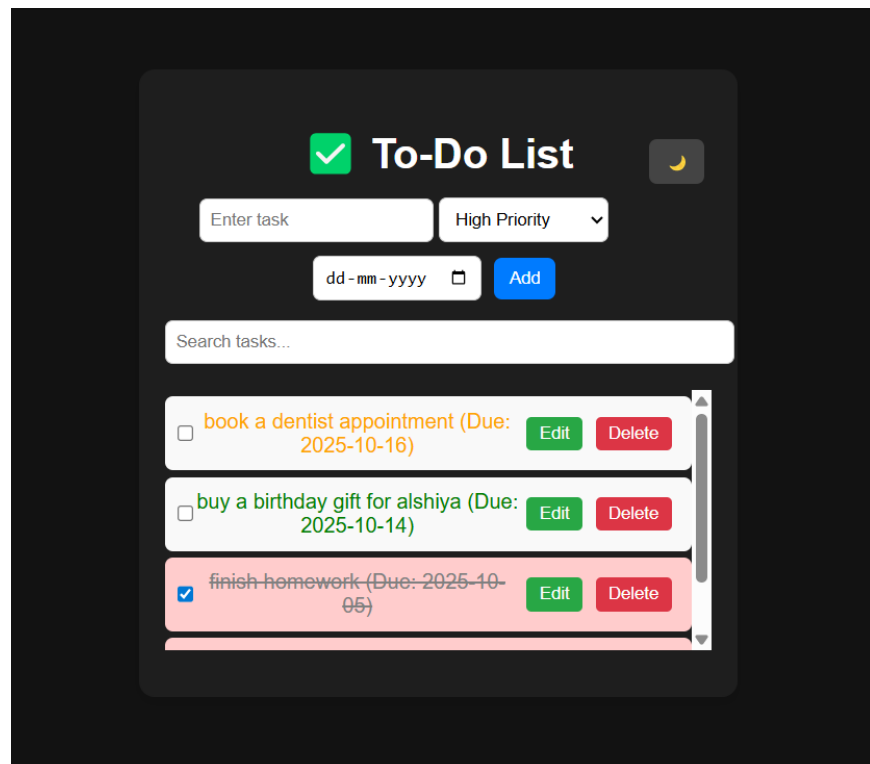


2. Added Tasks

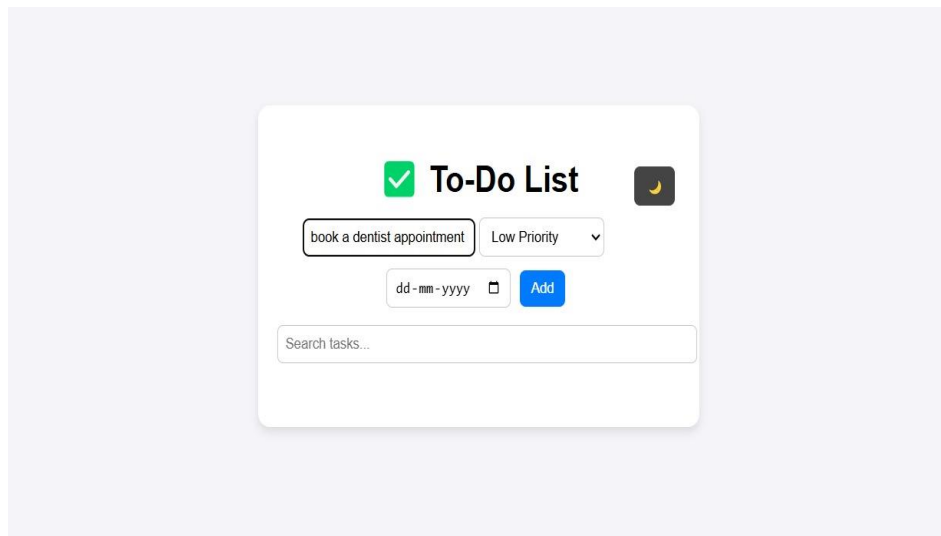


3.Dark/Light Mode

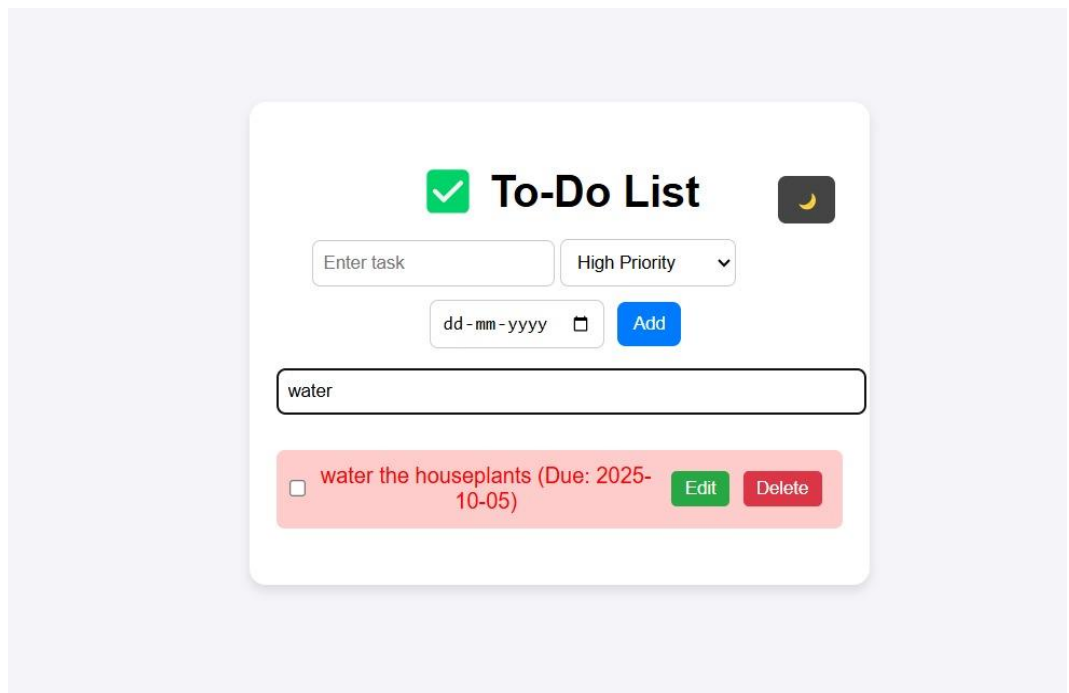
Dark Mode:



Light Mode:



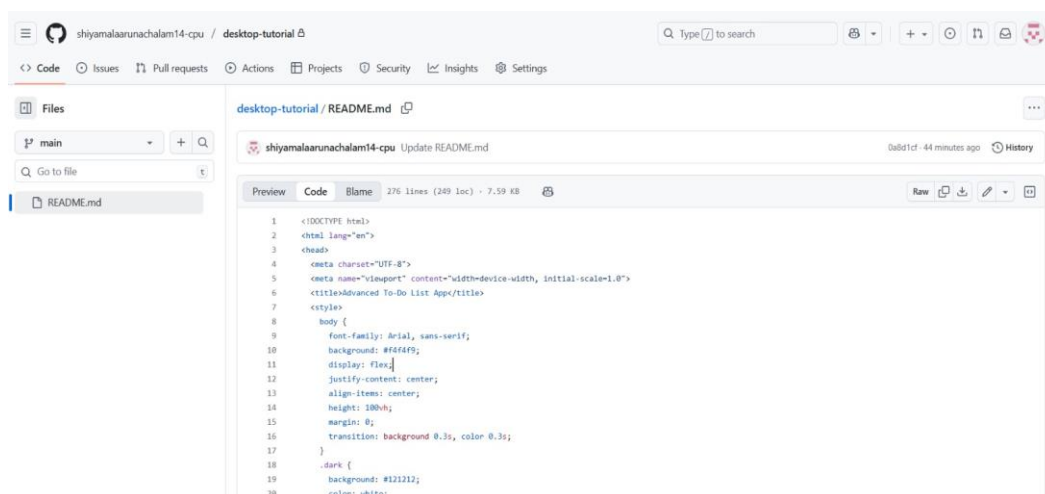
3.Search Functionality:

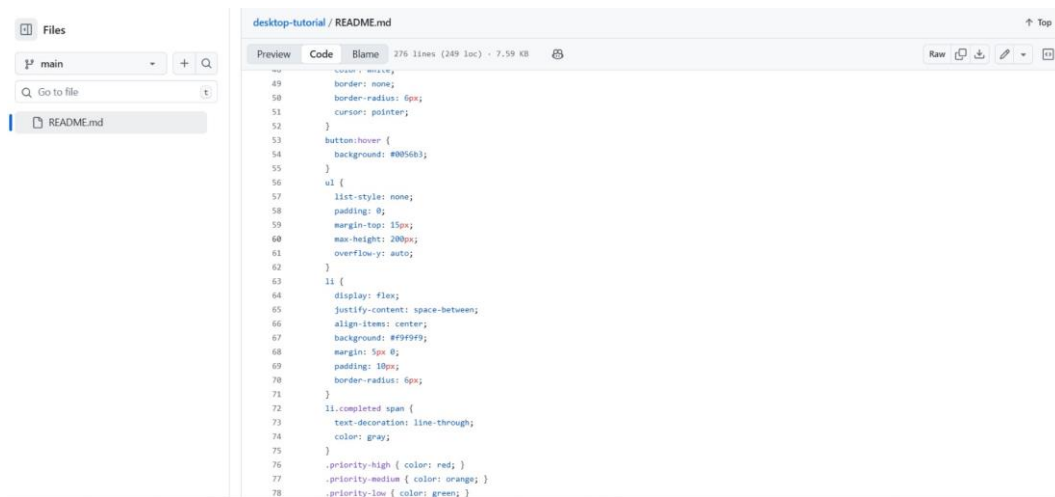
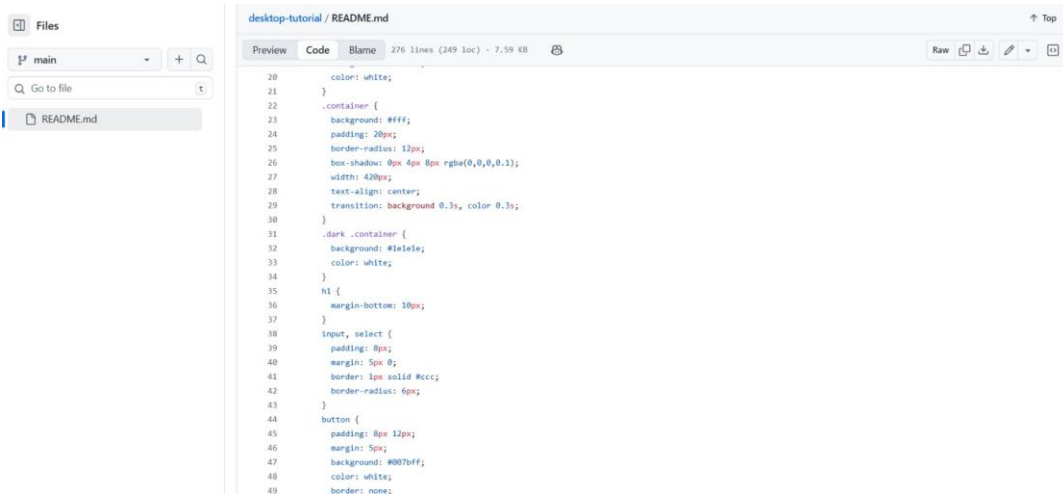


Challenges & Solutions

- **Challenge:** Making tasks persist after page refresh.
- **Solution:** Used `localStorage.setItem()` and `localStorage.getItem()`.
- **Challenge:** Designing responsive UI.
- **Solution:** Applied Bootstrap grid system.
- **Challenge:** Implementing search functionality
- **Solution:** Filtered tasks dynamically using JavaScript `includes()` function.
- **Challenge:** Theme color switching logic
- **Solution:** Used JS to toggle dark and light CSS classes

GitHub Readme





Files

main

Go to file

README.md

desktop-tutorial / README.md

PreviewCodeBlame276 lines (249 loc) · 7.59 KB

RawDownloadEditTop

```
78 .priority-low { color: green; }
79 .overdue { background: #ffcccc; }
80 .delete {
81     background: #dc3545;
82     color: white;
83     border: none;
84     padding: 5px 10px;
85     border-radius: 4px;
86     cursor: pointer;
87 }
88 .edit {
89     background: #28a745;
90     color: white;
91     border: none;
92     padding: 5px 10px;
93     border-radius: 4px;
94     cursor: pointer;
95     margin-right: 5px;
96 }
97 .toggle-btn {
98     background: #444;
99     color: white;
100     float: right;
101     margin-top: -40px;
102     margin-bottom: 10px;
103 }
104 .search-box {
105     width: 100%;
106     padding: 8px;
107     margin-top: 10px;
```

Files

main

Go to file

README.md

desktop-tutorial / README.md

PreviewCodeBlame276 lines (249 loc) · 7.59 KB

RawDownloadEditTop

```
136
137     if (taskInput.value.trim() === "") return;
138
139     let task = {
140         text: taskInput.value,
141         priority: priority,
142         dueDate: dueDate,
143         completed: false
144     };
145
146     saveTask(task);
147     displayTask(task);
148
149     taskInput.value = "";
150     document.getElementById("dueDate").value = "";
151 }
152
153 function displayTask(task) {
154     let li = document.createElement("li");
155
156     let checkbox = document.createElement("input");
157     checkbox.type = "checkbox";
158     checkbox.checked = task.completed;
159     checkbox.onchange = function () {
160         task.completed = checkbox.checked;
161         if (checkbox.checked) li.classList.add("completed");
162         else li.classList.remove("completed");
163         updateLocalStorage();
164     };
165 }
```

Files

main

Go to file

README.md

desktop-tutorial / README.md

PreviewCodeBlame276 lines (249 loc) · 7.59 KB

RawDownloadEditTop

```
166 let span = document.createElement("span");
167 span.textContent = task.text;
168 span.classList.add("priority-" + task.priority);
169
170 if (task.dueDate) {
171   let due = new Date(task.dueDate);
172   let today = new Date();
173   if (due < today && !task.completed) li.classList.add("overdue");
174   span.textContent += " (Due: " + task.dueDate + ")";
175 }
176
177 // Edit button
178 let editBtn = document.createElement("button");
179 editBtn.textContent = "Edit";
180 editBtn.className = "edit";
181 editBtn.onclick = function () {
182   let newText = prompt("Edit task:", task.text);
183   if (newText === null || newText.trim() === "") return;
184
185   let newPriority = prompt("Change priority (low, medium, high):", task.priority);
186   if (!["low", "medium", "high"].includes(newPriority)) newPriority = task.priority;
187
188   let newDate = prompt("Change due date (YYYY-MM-DD):", task.dueDate);
189
190   task.text = newText.trim();
191   task.priority = newPriority;
192   task.dueDate = newDate;
193
194   li.remove();
195   updateTask(task.text, task); // update by new text
```

Files

main

Go to file

README.md

desktop-tutorial / README.md

PreviewCodeBlame276 lines (249 loc) · 7.59 KB

RawDownloadEditTop

```
195 updateTask(task.text, task); // update by new text
196 displayTask(task);
197 updateLocalStorage();
198 };
199
200 let delBtn = document.createElement("button");
201 delBtn.textContent = "Delete";
202 delBtn.className = "delete";
203 delBtn.onclick = function () {
204   li.remove();
205   removeTask(task.text);
206 };
207
208 li.appendChild(checkbox);
209 li.appendChild(span);
210 li.appendChild(editBtn);
211 li.appendChild(delBtn);
212
213 if (task.completed) li.classList.add("completed");
214
215 document.getElementById("tasklist").appendChild(li);
216 }
217
218 function filterTasks() {
219   let searchValue = document.getElementById("search").value.toLowerCase();
220   let tasks = document.getElementById("tasklist").getElementsByTagName("li");
221   for (let i = 0; i < tasks.length; i++) {
222     let text = tasks[i].innerText.toLowerCase();
223     tasks[i].style.display = text.includes(searchValue) ? "" : "none";
224   }
225 }
```

```
224 }
225 }
226
227 function toggleDarkMode() {
228   document.body.classList.toggle("dark");
229 }
230
231 // Local Storage Functions
232 function saveTask(task) {
233   let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
234   tasks.push(task);
235   localStorage.setItem("tasks", JSON.stringify(tasks));
236 }
237
238 function loadTasks() {
239   let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
240   tasks.forEach(task => displayTask(task));
241 }
242
243 function removeTask(text) {
244   let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
245   tasks = tasks.filter(t => t.text !== text);
246   localStorage.setItem("tasks", JSON.stringify(tasks));
247 }
248
249 function updateTask(oldText, updatedTask) {
250   let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
251   tasks = tasks.map(t => (t.text === oldText ? updatedTask : t));
252   localStorage.setItem("tasks", JSON.stringify(tasks));
253 }
```

```
249 function updateTask(oldText, updatedTask) {
250   let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
251   tasks = tasks.map(t => (t.text === oldText ? updatedTask : t));
252   localStorage.setItem("tasks", JSON.stringify(tasks));
253 }
254
255 function updateLocalStorage() {
256   let tasks = [];
257   document.querySelectorAll("#tasklist li").forEach(li => {
258     let checkbox = li.querySelector("input[type='checkbox']");
259     let span = li.querySelector("span");
260     let taskText = span.textContent.split(" Due:")[0];
261     let priorityClass = span.className.split("-")[1];
262     let dueDateMatch = span.textContent.match(/\(Due: (.*)\)\)/);
263     let dueDate = dueDateMatch ? dueDateMatch[1] : "";
264
265     tasks.push({
266       text: taskText,
267       priority: priorityClass,
268       dueDate: dueDate,
269       completed: checkbox.checked
270     });
271   });
272   localStorage.setItem("tasks", JSON.stringify(tasks));
273 }
274 </script>
275 </body>
276 </html>
```

Final submission details

GitHub Repository :

<https://github.com/uchini2006/Uchini-Makali-A.git>

The final submission includes both the GitHub repository and deployed version of the project. All documentation, screenshots, and setup instructions are included for reference.